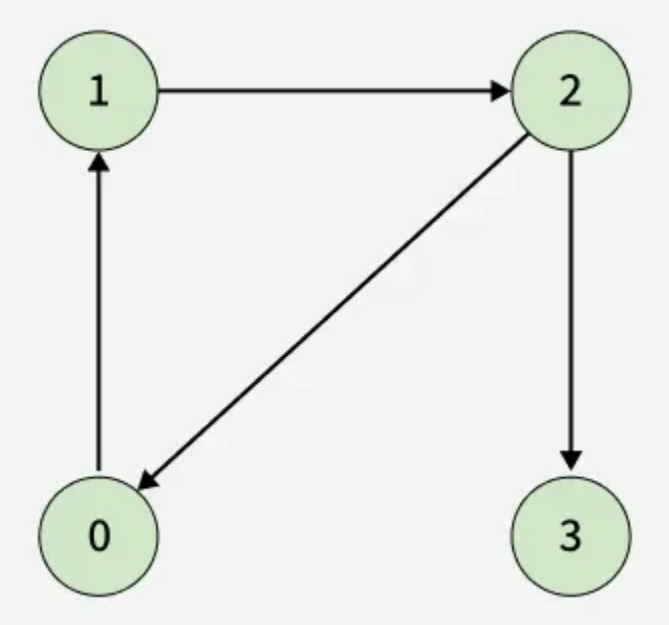
Detect Cycle in a Directed Graph

Last Updated : 04 Nov, 2025

Given a directed graph represented by its adjacency list **adj[][]**, determine whether the graph contains a cycle/Loop or not.
A cycle is a path that starts and ends at the same vertex, following the direction of edges.

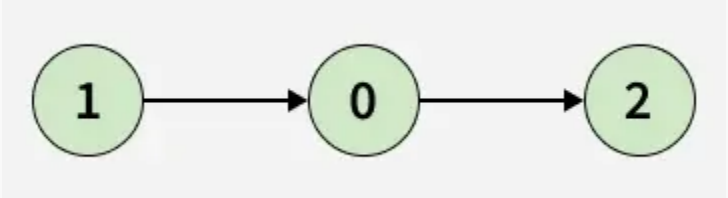
Examples:

Input: *adj[][] = [[1], [2], [0, 3], []]*



Output: *true*
Explanation: *There is a cycle 0 -> 1 -> 2 -> 0.*

Input: *adj[][] = [[2], [0], []]*



Output: *false*
Explanation: *There is no cycle in the graph.*

Table of Content

[Approach 1] Using DFS - O(V + E) Time and O(V) Space

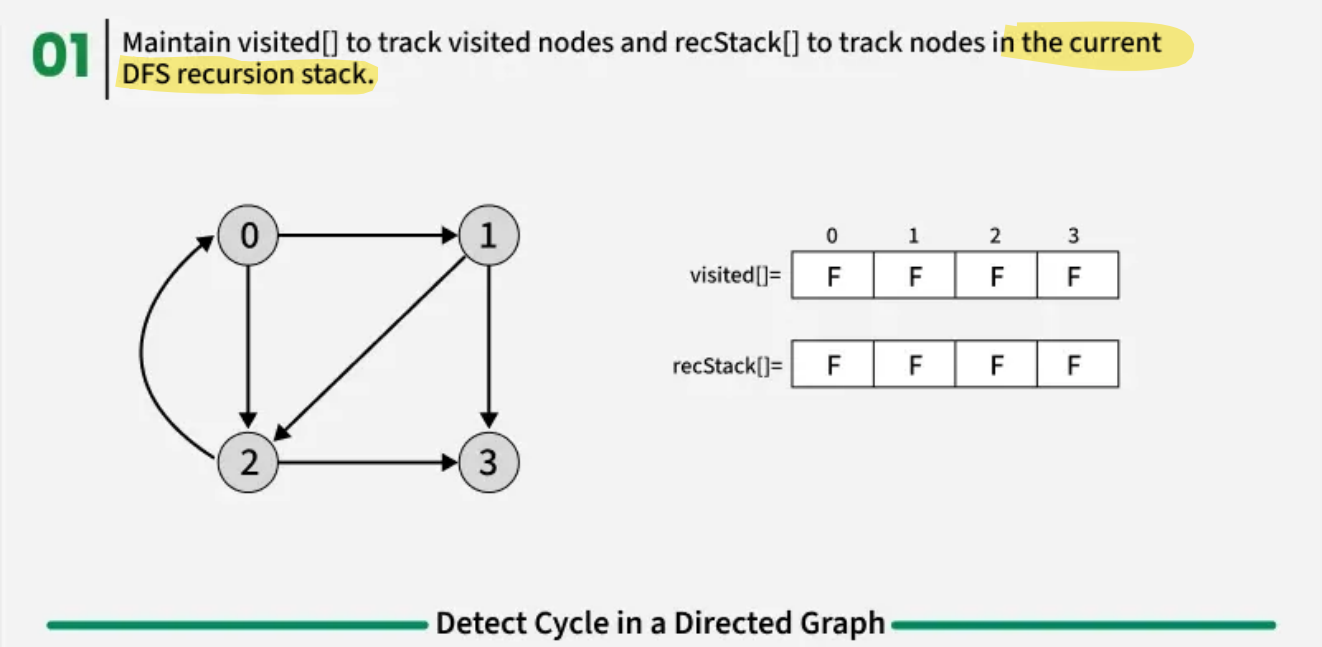
To detect a cycle in a directed graph, we use Depth First Search (DFS). In DFS, we go as deep as possible from a starting node. If during this process, we reach a node that we've already visited in the same DFS path, it means we've gone back to an ancestor — this shows a cycle exists.
But there's a problem: When we start DFS from one node, some nodes get marked as visited. Later, when we start DFS from another node, those visited nodes may appear again, even if there's no cycle.
So, using only visited[] isn't enough.

To fix this, we use two arrays:

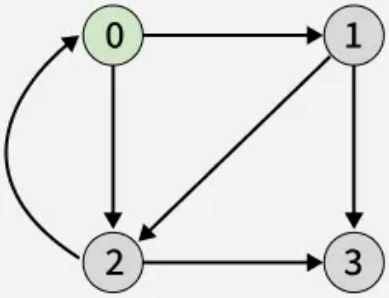
- *visited[] - marks nodes visited at least once.*
- *recStack[] - marks nodes currently in the recursion (active) path.*

If during DFS we reach a node that's already in the recStack, we've found a path from the current node back to one of its ancestors, forming a cycle. As soon as we finish exploring all paths from a node, we remove it from the recursion stack by marking recStack[node] = false. This ensures that only the nodes in the current DFS path are tracked.

Illustration:

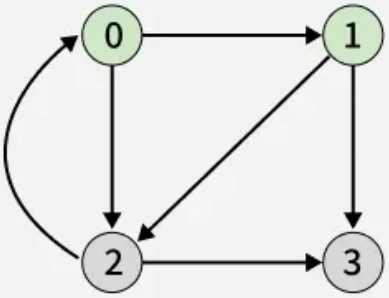


02 Start DFS from any unvisited node (e.g., node 0), mark it as visited, add it to the recursion stack, and recurse for all adjacent nodes.



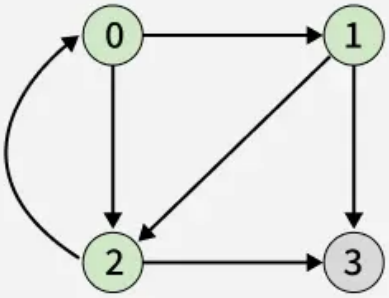
	0	1	2	3
visited[]=	T	F	F	F
recStack[]=	T	F	F	F

03 Node 1 is adjacent to node 0, traverse to node 1, call DFS, and mark it as visited and in the reStack



	0	1	2	3
visited[]=	T	T	F	F
recStack[]=	T	T	F	F

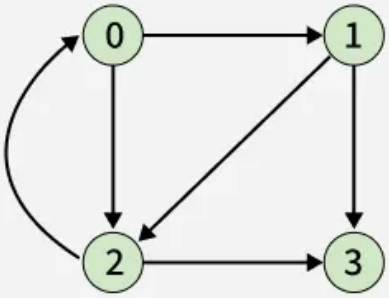
04 Node 2 is adjacent to node 1, traverse to node 2, call DFS, and mark it as visited and in the reStack



	0	1	2	3
visited[]=	T	T	T	F
recStack[]=	T	T	T	F

05

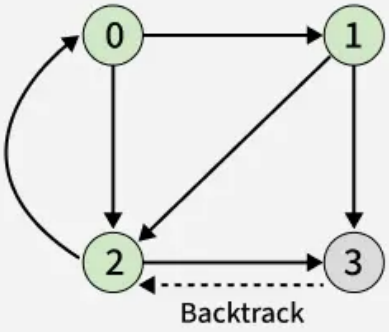
Node 3 is adjacent to node 2, traverse to node 3, call DFS, and mark it as visited and in the reStack



	0	1	2	3
visited[]=	T	T	T	T
recStack[]=	T	T	T	T

06

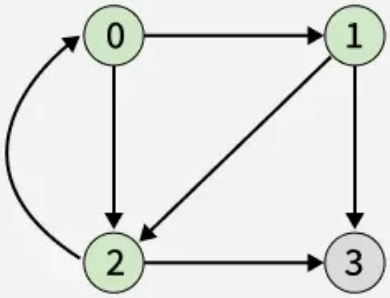
Since node 3 has no adjacent nodes to visit, backtrack and unmark it from recStack as It is no longer part of the current path.



	0	1	2	3
visited[]=	T	T	T	T
recStack[]=	T	T	T	F

07

While traversing node 2's adjacent node 0, it is already marked in recStack, confirming the presence of a cycle.



	0	1	2	3
visited[]=	T	T	T	T
recStack[]=	T	T	T	F

Cycle Found

```
#include <iostream>
#include <vector>
using namespace std;

//Driver Code Ends
// Utility DFS function to detect cycle in a directed graph
bool isCyclicUtil(vector<vector<int>>& adj, int u, vector<bool>& visited,
vector<bool>& recStack) {

    // node is already in recursion stack cycle found
    if (recStack[u]) return true;

    // already processed no need to visit again
    if (visited[u]) return false;

    visited[u] = true;
    recStack[u] = true;

    // Recur for all adjacent nodes
    for (int v : adj[u]) {
        if (isCyclicUtil(adj, v, visited, recStack))
            return true;
    }
    // remove from recursion stack before backtracking
    recStack[u] = false;
    return false;
}
```

Java Python C# JavaScript

```
// Utility DFS function to detect cycle in a directed graph
bool isCyclicUtil(vector<vector<int>>& adj, int u, vector<bool>&
visited,
vector<bool>& recStack) {

     // node is already in recursion stack cycle found
    if (recStack[u]) return true;

     // already processed no need to visit again
    if (visited[u]) return false;

    visited[u] = true;
    recStack[u] = true;

    // Recur for all adjacent nodes
    for (int v : adj[u]) {
        if (isCyclicUtil(adj, v, visited, recStack))
            return true;
    }
     // remove from recursion stack before backtracking
    recStack[u] = false;
    return false;
}

// Function to detect cycle in a directed graph
bool isCyclic(vector<vector<int>>& adj) {
    int V = adj.size();
    vector<bool> visited(V, false);
    vector<bool> recStack(V, false);

    // Run DFS from every unvisited node
    for (int i = 0; i < V; i++) {
        if (!visited[i] && isCyclicUtil(adj, i, visited,
recStack))
            return true;
    }
    return false;
}
```

```
int main() {
    vector<vector<int>> adj = {{1},{2},{0, 3}};

    cout << (isCyclic(adj) ? "true" : "false") << endl;
    return 0;
}
```

Output

```
true
```

[Approach 2] Using Topological Sorting - O(V + E) Time and O(V) Space

The idea is to use Kahn's algorithm because it works only for Directed Acyclic Graphs (DAGs). So, while performing topological sorting using Kahn's algorithm, if we are able to include all the vertices in the topological order, it means the graph has no cycle and is a DAG.
However, if at the end there are still some vertices left (i.e., their in-degree never becomes 0), it means those vertices are part of a cycle. Hence, if we cannot get all the vertices in the topological sort, the graph must contain at least one cycle.

C++ Java Python C# JavaScript

```cpp
bool isCyclic(vector<vector<int>> &adj)
{
    int V = adj.size();
    // Array to store in-degree of each vertex
    vector<int> inDegree(V, 0);
    queue<int> q;
    // Count of visited (processed) nodes
    int visited = 0;

    //Compute in-degrees of all vertices
    for (int u = 0; u < V; ++u)
    {
        for (int v : adj[u])
        {
            inDegree[v]++;
        }
    }

    //  Add all vertices with in-degree 0 to the queue
```

```
    for (int u = 0; u < V; ++u)
    {
        if (inDegree[u] == 0)
        {
            q.push(u);
        }
    }

    // Perform BFS (Topological Sort)
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        visited++;

        // Reduce in-degree of neighbors
        for (int v : adj[u])
        {
            inDegree[v]--;
            if (inDegree[v] == 0)
            {
                // Add to queue when in-degree becomes 0
                q.push(v);
            }
        }
    }

    //  If visited nodes != total nodes, a cycle exists
    return visited != V;
}
```
↔

Output

```
true
```

Similar Article:

[Detect Cycle in a direct graph using colors]

Detect Cycle in a Directed graph using colors